

Payment.java - Code Structure & Links

1. CLASS DEFINITION

```
public abstract class Payment
```

Explanation: This is the Abstract Base Class. It is the blueprint for all payment types in the system. The "abstract" keyword means you cannot create a direct instance of "Payment". It forces all child classes (like CreditCardPayment) to link to it and inherit its structure.

2. CLASS VARIABLES (FIELDS)

```
private static int ticketCounter = 1000;
```

Explanation: A "static" variable belongs to the class itself, not individual objects. It acts as a global counter shared across every payment to ensure each transaction gets a unique ID.

```
protected String paymentId;  
protected double amount;  
protected String currency;  
protected String status;  
protected LocalDateTime timestamp;
```

Explanation: The "protected" keyword provides Encapsulation. It hides these variables from outside classes (like PaymentProcessor) to prevent accidental tampering, while keeping them fully accessible to linked child classes.

3. CONSTRUCTOR

```
public Payment(double amount, String currency)
```

Explanation: This constructor automatically runs whenever a child class is created. It validates the data (throws an error if amount <= 0), automatically generates a unique paymentId using the ticketCounter, sets the status to "PENDING", and captures the exact time the object was created.

4. ABSTRACT METHODS

```
public abstract boolean process(BiConsumer<String, String> log);
```

Explanation: This is the core architectural contract of the program. Because it is marked "abstract", it contains no actual code. It strictly forces every linked child class to write their own implementation of the process() method. The BiConsumer parameter acts as a pipeline to pass log messages back to the UI.